

Étude de la complexité d'une plateforme de planification basée sur DSL et MiniZinc

LOMOFOUET NDONGMO H. Jauress

27 avril 2026

Résumé

Ce rapport étudie la complexité d'une plateforme de planification construite autour d'un DSL et d'une génération vers MiniZinc. L'objectif est d'expliquer, de manière théorique et méthodologique, pourquoi certaines instances se résolvent rapidement alors que d'autres deviennent coûteuses ou conduisent à des *timeouts*. L'analyse distingue explicitement : (i) la complexité logicielle du pipeline (parsing, AST, génération, flattening, appel solveur, restitution), (ii) la complexité combinatoire intrinsèque du problème de planification.

Le document fournit des définitions formelles, des propositions avec preuves, des bornes asymptotiques, une analyse de la taille du modèle généré, une borne naïve de l'espace de recherche, une preuve de NP-difficulté par réduction depuis la coloration de graphe, et une discussion comparative des solveurs (HiGHS, Gecode, Chuffed, OR-Tools CP-SAT). Les résultats expérimentaux n'étant pas fournis, une méthodologie complète de mesure est proposée, avec des tableaux LaTeX prêts à être remplis.

Table des matières

1	Introduction	4
1.1	Objectif général du rapport	4
1.2	Deux niveaux de complexité à distinguer	4
1.3	Cadre méthodologique	4
1.4	Notions cruciales de complexité à retenir	4
2	Rappels sur la complexité algorithmique	5
2.1	Définitions fondamentales	5
2.2	Pire cas, meilleur cas, cas moyen	5
2.3	Croissances usuelles	5
2.4	Complexité théorique vs temps observé	5
3	Architecture de la plateforme	5
3.1	Pipeline formel	5
3.2	Entrées, sorties et coûts par étape	6
3.3	Remarque sur le flattening	6
4	Paramètres de taille d'une instance	7
4.1	Notations	7
4.2	Rôle des paramètres	7
5	Taille du modèle MiniZinc généré	7
5.1	Variables principales	7
5.2	Exemple numérique	8
6	Analyse de la complexité temporelle	8
6.1	Nombre de contraintes générées par famille	8
6.2	Contraintes de rôle	9
6.3	Contraintes d'affectation	9
6.4	Contraintes de non-chevauchement	10
6.5	Contraintes par slot	10
6.6	Contraintes de précedence	10

6.7	Contraintes de préférences	10
6.8	Taille du modèle généré et coût de génération	10
7	Analyse de la complexité spatiale	11
7.1	Mémoire avant solveur	11
7.2	Discussion pratique	11
8	Complexité théorique du problème de planification	11
8.1	Espace de recherche naïf	11
8.2	NP-difficulté par réduction depuis la coloration	12
8.3	Optimisation vs satisfaction	12
9	Influence du choix du solveur	12
9.1	Principe général	12
9.2	Lecture pratique	13
10	Score de complexité et prédiction du temps	13
10.1	Pourquoi le temps exact est difficile à prédire	13
10.2	Exemples conceptuels	13
10.3	Score simple de complexité	14
10.4	Score pondéré	14
10.5	Classification de risque (seuils symboliques)	14
10.6	Modèle empirique de prédiction	14
11	Limites de l'approche	14
11.1	Limites théoriques	14
11.2	Limites pratiques	15
12	Conclusion	15
13	Récapitulatif des résultats démontrés	15
14	Références	16

1 Introduction

1.1 Objectif général du rapport

L’objectif est d’analyser la complexité d’une plateforme de planification basée sur un DSL, afin de comprendre pourquoi certaines instances sont faciles à résoudre tandis que d’autres sont lentes ou conduisent à un *timeout*. Le temps total peut être décomposé en :

$$T_{\text{total}} = T_{\text{parse}} + T_{\text{AST}} + T_{\text{generation}} + T_{\text{flattening}} + T_{\text{resolution}} + T_{\text{output}}. \quad (1)$$

Dans la pratique, la composante dominante est généralement

$$T_{\text{resolution}}, \quad (2)$$

car la recherche combinatoire est souvent le goulet d’étranglement.

1.2 Deux niveaux de complexité à distinguer

1. **Complexité de la plateforme logicielle** : parsing, construction d’AST, génération MiniZinc, lancement solveur, lecture de la sortie.
2. **Complexité du problème combinatoire** : affectations activités-créneaux, activités-ressources, rôles, respect des contraintes, optimisation des préférences.

1.3 Cadre méthodologique

Ce rapport distingue explicitement trois types d’énoncés :

- **Résultats théoriques démontrés** (propositions + preuves) ;
- **Estimations asymptotiques** (ordres de grandeur) ;
- **Observations empiriques** (à mesurer expérimentalement, sans résultats inventés).

1.4 Notions cruciales de complexité à retenir

- **Distinction structurelle** : la génération du modèle est typiquement polynomiale, la résolution peut être exponentielle.
- **Point dominant** : dans la plupart des cas, $T_{\text{resolution}}$ domine le temps total.
- **Explosion combinatoire** : les familles en $O(RA^2)$ (notamment non-chevauchement) et $O(RTA)$ (contraintes par slot) peuvent croître très vite.
- **Espace de recherche** : la borne naïve $T^A \times 2^{AR} \times 2^{AKR}$ explique la difficulté potentielle.
- **Théorie de la difficulté** : sous hypothèses raisonnables, la faisabilité est NP-difficile.
- **Optimisation** : prouver l’optimalité est au moins aussi difficile que trouver une solution faisable.
- **Prédiction** : un score de complexité permet une estimation de risque, pas une garantie de temps exact.

2 Rappels sur la complexité algorithmique

2.1 Définitions fondamentales

Définition 2.1 (Complexité temporelle). *La complexité temporelle d'un algorithme est la fonction qui associe à la taille de l'entrée le nombre d'opérations élémentaires nécessaires à son exécution.*

Définition 2.2 (Complexité spatiale). *La complexité spatiale d'un algorithme est la quantité de mémoire supplémentaire requise en fonction de la taille de l'entrée.*

Définition 2.3 (Notations asymptotiques). *Soient deux fonctions positives f et g .*

- $f(n) \in O(g(n))$ s'il existe $c > 0$ et n_0 tels que $f(n) \leq cg(n)$ pour tout $n \geq n_0$.
- $f(n) \in \Omega(g(n))$ s'il existe $c > 0$ et n_0 tels que $f(n) \geq cg(n)$ pour tout $n \geq n_0$.
- $f(n) \in \Theta(g(n))$ si $f(n) \in O(g(n))$ et $f(n) \in \Omega(g(n))$.

2.2 Pire cas, meilleur cas, cas moyen

- **Pire cas** : borne supérieure sur toutes les entrées de taille n .
- **Meilleur cas** : coût minimal sur les entrées de taille n .
- **Cas moyen** : espérance du coût sur une distribution d'entrées.

2.3 Croissances usuelles

Ordres de grandeur classiques : $O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, $O(n!)$. Les classes exponentielles et factorielles deviennent rapidement impraticables.

TABLE 1: Comparaison qualitative de croissances

n	$O(n)$	$O(n^2)$	$O(2^n)$	$O(n!)$
5	5	25	32	120
10	10	100	1024	3 628 800
20	20	400	1 048 576	2.43×10^{18}
30	30	900	1.07×10^9	2.65×10^{32}

2.4 Complexité théorique vs temps observé

La complexité asymptotique ne capture ni les constantes cachées, ni l'implémentation, ni l'architecture matérielle, ni les heuristiques du solveur. Elle fournit un cadre de comparaison, pas une prédiction exacte en secondes.

3 Architecture de la plateforme

3.1 Pipeline formel

Le pipeline considéré est :

DSL $\rightarrow$ Parser $\rightarrow$ AST $\rightarrow$ Générateur MiniZinc $\rightarrow$ Modèle MiniZinc $\rightarrow$
 Flattening $\rightarrow$ FlatZinc/Représentation interne $\rightarrow$ Solveur $\rightarrow$ Solution $\rightarrow$
 Planning exploitable

3.2 Entrées, sorties et coûts par étape

TABLE 2: Analyse des étapes du pipeline

Étape	Entrée	Sortie	Complexité temporelle (approx.)	Complexité spatiale (approx.)	Remarque
Parsing	Fichier DSL de taille L	AST	$O(L)$ (ou $O(L \log L)$ selon méthode)	$O(L)$	Dépend du parseur
Construction AST	Tokens/règles	AST enrichi	$O(L)$	$O(L)$	Structures syntaxiques
Génération MiniZinc	AST	Fichier MZN de taille M	$O(M)$	$O(M)$	Écriture textuelle
Flattening	MZN	FlatZinc/intern	variable, souvent polynomial mais dépendante du modèle	variable	Pré-traitement solveur
Résolution	FlatZinc/intern	solution/infaisable	potentiellement exponentielle (pire cas)	élevée possible	Étape dominante
Sortie	Solution solveur	planning exploitable	$O(\text{solution})$ à $O(M)$	faible à modérée	Sérialisation

3.3 Remarque sur le flattening

Le coût du flattening dépend fortement des transformations (décomposition de contraintes globales, expansion d'indices, linéarisation éventuelle). Il est donc préférable de le mesurer empiriquement en plus de l'estimer.

4 Paramètres de taille d'une instance

4.1 Notations

On introduit les paramètres :

A := nombre d'activités,
 R := nombre total de ressources,
 K := nombre de rôles,
 D := nombre de jours,
 S := nombre de slots par jour,
 $T := D \times S$ (nombre total de slots),
 C := nombre total de contraintes explicites DSL,
 P := nombre de préférences / contraintes souples,
 Q := nombre de précédences,
 W := nombre de fenêtres temporelles,
 E := nombre de contraintes d'exclusion/non-chevauchement,
 G := nombre de groupes/types de ressources (si utilisé),
 L := taille du DSL (caractères ou tokens).

4.2 Rôle des paramètres

- Paramètres influençant surtout la génération : L , C , structure de l'AST, schéma de traduction.
- Paramètres influençant fortement la résolution : A , R , K , T , P , Q , densité des interactions.

Les paramètres critiques sont généralement A , R , K , T , P :

- A : de nombreuses familles de contraintes croissent en A^2 ;
- R : multiplication des affectations ;
- K : multiplication des variables de rôle ;
- T : croissance des choix temporels ;
- P : coût supplémentaire de l'optimisation.

5 Taille du modèle MiniZinc généré

5.1 Variables principales

Dans le schéma considéré :

- `start_time[a]` : une variable de début par activité ;
- `assignment[a,r]` : booléen activité-ressource ;
- `role_assignment[a,k,r]` : booléen activité-rôle-ressource.

Proposition 5.1 (Nombre de variables `start_time`). $N_{start} = A$.

Démonstration. Par définition du modèle, il existe exactement une variable `start_time` pour chaque activité. Comme l'ensemble des activités a cardinal A , le nombre total est A . $\square$

Proposition 5.2 (Nombre de variables assignment). $N_{assign} = A \times R$.

Démonstration. Une variable `assignment[a,r]` est créée pour chaque couple (a, r) avec $a \in \{1, \dots, A\}$ et $r \in \{1, \dots, R\}$. Le nombre de couples est le produit des cardinaux : $A \times R$. $\square$

Proposition 5.3 (Nombre de variables role_assignment). $N_{role} = A \times K \times R$.

Démonstration. Une variable `role_assignment[a,k,r]` est créée pour chaque triplet (a, k, r) , avec a activité, k rôle, r ressource. Le nombre total de triplets vaut $A \times K \times R$. $\square$

Proposition 5.4 (Nombre total de variables principales).

$$N_{variables} = A + A \times R + A \times K \times R. \quad (3)$$

Asymptotiquement, $N_{variables} \in O(AKR)$ lorsque A, R, K varient.

Démonstration. Le total est la somme de trois familles disjointes : `start_time`, `assignment`, `role_assignment`. En additionnant les trois cardinalités déjà démontrées :

$$N_{variables} = N_{start} + N_{assign} + N_{role} = A + AR + AKR.$$

Comme AKR domine asymptotiquement lorsque les trois paramètres croissent, la borne $O(AKR)$ suit. $\square$

5.2 Exemple numérique

Pour $A = 30$, $R = 10$, $K = 3$:

$$\begin{aligned} N_{start} &= 30, \\ N_{assign} &= 30 \times 10 = 300, \\ N_{role} &= 30 \times 3 \times 10 = 900, \\ N_{variables} &= 30 + 300 + 900 = 1230. \end{aligned}$$

6 Analyse de la complexité temporelle

6.1 Nombre de contraintes générées par famille

TABLE 3: Estimations du nombre de contraintes par famille

Famille	Rôle	Paramètres	Nombre estimé	Asymptotique
Domaine activités	Borner domaines	dates, A, T	$\sim A$ à $\sim A \log T$ selon encodage	$O(A)$ à $O(AT)$

Famille	Rôle	Paramètres	Nombre estimé	Asymptotique
Disponibilité ressources	Interdire res- sources indispo- nibles	A, R, W, T	dépend de granu- larité temporelle	souvent $O(ARW)$ ou $O(ART)$
Affectation activité-ressource	Lier activités et ressources	A, R	$A \times R$	$O(AR)$
Rôles	Affecter rôle par activité/res- source	A, K, R	$A \times K \times R$	$O(AKR)$
Cardinalité	Nombre min/- max de res- sources	A, R	$\sim A$ à $\sim AR$	$O(A)+O(AR)$
Capacité	Limites cumu- lées	A, R, T	dépend modélisa- tion	souvent $O(ART)$
Non- chevauchement	Empêcher conflits tempo- rels	A, R	$R \times \binom{A}{2}$	$O(RA^2)$
Précédence	Ordres partiels	Q ou A_1, A_2	Q ou A_1A_2	$O(Q)$ ou $O(A_1A_2)$
Avant/Après temporel	Fenêtres, délais, lags	A, W	$\sim W$ à $\sim AW$	$O(W)$ + $O(AW)$
Par slot	Vérifications par créneau	R, T, A	$R \times T \times A$	$O(RTA)$
Préférences/pénalités	Objectif souple	P	P	$O(P)$ (struc- ture), impact solveur > li- néaire possible

6.2 Contraintes de rôle

Proposition 6.1. *Si chaque activité peut mobiliser K rôles et R ressources, alors le nombre de relations de rôle est $A \times K \times R$, donc en $O(AKR)$.*

Démonstration. Chaque relation est indexée par un triplet (a, k, r) . Le nombre de triplets possibles est le produit des cardinalités, soit $A \cdot K \cdot R$. La borne asymptotique est $O(AKR)$. $\square$

6.3 Contraintes d'affectation

Proposition 6.2. *Le nombre de contraintes d'affectation activité-ressource est $A \times R$, donc en $O(AR)$.*

Démonstration. Une contrainte (ou une famille d'atomes linéaires/logiques équivalente) est associée à chaque couple (a, r) . Le nombre total de couples est AR . $\square$

6.4 Contraintes de non-chevauchement

Proposition 6.3. *Si le modèle impose une contrainte de non-chevauchement pour chaque paire d'activités et pour chaque ressource, alors le nombre de contraintes de non-chevauchement est*

$$N_{\text{overlap}} = R \times \frac{A(A-1)}{2}, \quad (4)$$

et donc $N_{\text{overlap}} \in O(RA^2)$.

Démonstration. Le nombre de paires distinctes d'activités vaut $\binom{A}{2} = A(A-1)/2$. Pour chaque ressource, il faut imposer la non-superposition sur chacune de ces paires. En multipliant par R :

$$N_{\text{overlap}} = R \binom{A}{2} = R \frac{A(A-1)}{2}.$$

Comme $A(A-1)/2 \in \Theta(A^2)$, on obtient $N_{\text{overlap}} \in O(RA^2)$. $\square$

Remarque 6.1. *Cette famille est critique : si A double, le terme quadratique est multiplié approximativement par 4 (à R fixé).*

6.5 Contraintes par slot

Si une vérification est réalisée pour chaque triplet (r, t, a) , la taille est $R \times T \times A$, donc $O(RTA)$. Cette famille devient coûteuse quand $T = D \times S$ croît (horizon long ou granularité fine).

6.6 Contraintes de précédence

Si les précédences sont explicitement données sous forme de Q arcs, la taille est $O(Q)$. En revanche, si elles sont générées entre deux ensembles d'activités A_1 et A_2 , on peut atteindre $O(A_1 A_2)$.

6.7 Contraintes de préférences

Le nombre de contraintes de préférence est souvent $O(P)$. Cependant, leur impact sur $T_{\text{resolution}}$ peut être majeur : le solveur doit explorer et prouver l'optimalité, pas seulement trouver un point faisable.

6.8 Taille du modèle généré et coût de génération

On note M la taille (textuelle/structurelle) du modèle MiniZinc généré.

Proposition 6.4 (Taille approximative du modèle). *Sous les hypothèses d'encodage usuelles ci-dessus,*

$$M = O(A + AR + AKR + RA^2 + RTA + Q + P). \quad (5)$$

Démonstration. M est la somme des contributions des familles de variables et de contraintes. Les ordres dominants déjà établis sont : A , AR , AKR , RA^2 , RTA , Q , P . La somme de ces termes donne la borne annoncée. $\square$

Proposition 6.5 (Complexité temporelle de la génération).

$$T_{\text{generation}} \in O(M). \quad (6)$$

Démonstration. La génération consiste à parcourir des structures intermédiaires et écrire des fragments textuels de variables/contraintes. Le coût unitaire est proportionnel à la taille des fragments produits. En agrégeant sur l'ensemble des éléments générés, le coût total est proportionnel à la taille totale produite M . $\square$

Remarque 6.2. *Cette étape est en général polynomiale en les paramètres d'entrée, contrairement à la résolution qui peut être exponentielle en pire cas.*

7 Analyse de la complexité spatiale

7.1 Mémoire avant solveur

Avant résolution, la mémoire inclut : fichier DSL, AST, structures intermédiaires, modèle MiniZinc, et éventuellement buffers de sortie.

Proposition 7.1 (Complexité spatiale de génération). *Une estimation est*

$$S_{\text{generation}} = O(L + A + R + K + T + M). \quad (7)$$

En pratique, M domine souvent, donc $S_{\text{generation}} \in O(M)$.

Démonstration. Chaque composant occupe un volume au plus linéaire dans sa taille logique : le DSL en L , les métadonnées de taille A, R, K, T , puis le modèle produit en M . La somme des contributions fournit la borne. Lorsque M est significativement plus grand que les autres termes, il devient dominant. $\square$

7.2 Discussion pratique

La mémoire du solveur n'est pas incluse dans $S_{\text{generation}}$ et peut être substantielle (structures de propagation, nœuds de recherche, coupes, clauses apprises selon solveur).

8 Complexité théorique du problème de planification

8.1 Espace de recherche naïf

Proposition 8.1 (Borne naïve de l'espace de recherche). *Si chaque activité choisit un slot parmi T , chaque couple activité-ressource est binaire, et chaque triplet activité-rôle-ressource est binaire, alors :*

$$E_{\text{search}} = T^A \times 2^{AR} \times 2^{AKR} = T^A \times 2^{AR(K+1)}. \quad (8)$$

Démonstration. Par produit cartésien des choix bruts :

- temps : T choix par activité, soit T^A affectations ;
- affectations activité-ressource : AR booléens indépendants en borne naïve, soit 2^{AR} ;
- affectations rôle : AKR booléens, soit 2^{AKR} .

Le nombre total d'états bruts est le produit des cardinalités de ces ensembles de choix, d'où la formule. $\square$

Remarque 8.1. *Cette borne est pessimiste : la propagation, les relaxations, le branch-and-bound, le branch-and-cut, l'apprentissage de clauses, les heuristiques et les détections d'infaisabilité réduisent fortement l'espace effectivement exploré.*

8.2 NP-difficulté par réduction depuis la coloration

Proposition 8.2 (NP-difficulté). *Sous les hypothèses suivantes :*

- *le DSL exprime des contraintes d'incompatibilité/non-simultanéité ;*
- *les slots sont discrets ;*
- *les activités ont une durée compatible avec une occupation d'un slot ;*
- *une solution candidate se vérifie en temps polynomial ;*

la version décisionnelle du problème de faisabilité du planning est NP-difficile. De plus, elle est NP-complète si la vérification est polynomiale.

Démonstration. On réduit le problème de k -coloration (NP-complet pour $k \geq 3$) au problème de planification.

Soit un graphe $G = (V, E)$ et un entier $k \geq 3$.

- Chaque sommet $v \in V$ devient une activité a_v .
- Les k couleurs deviennent k slots temporels.
- Chaque activité doit être placée dans exactement un slot.
- Pour chaque arête $(u, v) \in E$, on impose la contrainte : a_u et a_v ne peuvent pas être dans le même slot.

Si G est k -coloriable, on affecte à chaque activité le slot correspondant à sa couleur : toutes les contraintes sont satisfaites, donc le planning est faisable.

Réciproquement, si un planning faisable existe, l'affectation des slots aux activités définit une couleur par sommet, et les contraintes sur les arêtes garantissent des couleurs différentes aux extrémités : on obtient une k -coloration valide.

La transformation est polynomiale en $|V| + |E|$. Donc la faisabilité de planning est au moins aussi difficile que la k -coloration : elle est NP-difficile. Si la vérification d'une solution est polynomiale, le problème appartient à NP, donc il est NP-complet. $\square$

8.3 Optimisation vs satisfaction

Proposition 8.3. *Le problème d'optimisation de planning est au moins aussi difficile que le problème de faisabilité correspondant.*

Démonstration. Toute instance de faisabilité peut être transformée en instance d'optimisation en ajoutant un objectif constant (ou neutre). Un solveur d'optimisation qui retourne une solution optimale permet alors de décider l'existence d'une solution faisable. Donc un algorithme pour l'optimisation résout la faisabilité : l'optimisation est au moins aussi difficile. $\square$

9 Influence du choix du solveur

9.1 Principe général

Aucun solveur n'est universellement meilleur : les performances dépendent de la représentation MiniZinc obtenue après flattening.

TABLE 4: Comparaison qualitative de solveurs

Solveur	Type	Modèle favorable	Points forts	Risques / limites
HiGHS	LP/MIP	Formulations linéaires (continues, entières, binaires)	Très bon sur MILP structurés, coupes et bornes efficaces	Moins naturel pour contraintes CP globales
Gecode	CP (do-maines finis)	Contraintes globales, propagation forte	Bon contrôle de recherche CP, robuste sur CSP structurés	Peut être moins favorable si modèle essentiellement booléen linéarisé massif
Chuffed	CP + LCG	Modèles combinatoires avec implications et conflits utiles à apprendre	Apprentissage de clauses, bonne performance sur certains CP difficiles	Sensible à la formulation et au branching
OR-Tools CP-SAT	Hybride CP/SAT/-MIP	Beaucoup de booléens + linéaire entier + implications	Très performant sur nombreux modèles combinatoires modernes	Comportement dépendant de l'encodage, parfois instable selon structure

9.2 Lecture pratique

Le choix du solveur doit être guidé par la forme réelle du modèle aplati, pas seulement par le DSL d'origine.

10 Score de complexité et prédiction du temps

10.1 Pourquoi le temps exact est difficile à prédire

Deux instances ayant mêmes (A, R, K, T) peuvent avoir des temps très différents. Facteurs : densité de contraintes, nombre de solutions, faisabilité vs infaisabilité, coût de preuve d'infaisabilité, présence d'objectif, symétries, qualité d'encodage, stratégie de recherche, solveur, bornes de domaines, contraintes trop faibles ou trop fortes.

Remarque 10.1. *Différences conceptuelles importantes :*

- trouver une solution faisable ;
- prouver l'absence de solution ;
- prouver l'optimalité d'une solution.

Ces trois tâches peuvent avoir des coûts radicalement différents.

10.2 Exemples conceptuels

Instance potentiellement facile : ressources abondantes, nombreux slots, peu de contraintes, nombreuses solutions.

Instance potentiellement difficile : ressources rares, fenêtres serrées, incompatibilités nombreuses, préférences à optimiser, solution rare ou absente.

10.3 Score simple de complexité

On propose un indicateur structurel :

$$\text{Score}_{\text{simple}} = A + AR + AKR + RA^2 + RTA + Q + P. \quad (9)$$

Interprétation des termes : activités, affectations, rôles, non-chevauchement, contraintes par slot, précédences, préférences.

10.4 Score pondéré

$$\text{Score} = w_1A + w_2AR + w_3AKR + w_4RA^2 + w_5RTA + w_6Q + w_7P, \quad (10)$$

avec $(w_1, \dots, w_7)$ appris/ajustés expérimentalement par solveur et par famille d'instances.

10.5 Classification de risque (seuils symboliques)

TABLE 5: Classification symbolique du risque de lenteur

Classe	Condition symbolique	Interprétation
Faible	$\text{Score} < \theta_1$	Résolution probablement rapide
Moyen	$\theta_1 \leq \text{Score} < \theta_2$	Résolution probablement acceptable
Élevé	$\theta_2 \leq \text{Score} < \theta_3$	Risque de lenteur
Très élevé	$\text{Score} \geq \theta_3$	Risque de timeout

10.6 Modèle empirique de prédiction

On peut modéliser : $T_{\text{solve}} \approx f(\text{Score})$, ou utiliser une régression :

$$\begin{aligned} \log(T_{\text{solve}} + 1) = & \beta_0 + \beta_1A + \beta_2R + \beta_3K + \beta_4T + \beta_5(AR) \\ & + \beta_6(AKR) + \beta_7(RA^2) + \beta_8(RTA) + \beta_9Q + \beta_{10}P. \end{aligned} \quad (11)$$

L'usage de $\log(T_{\text{solve}} + 1)$ se justifie par la forte dispersion des temps (ms à minutes), la stabilisation des écarts et une meilleure modélisation des ordres de grandeur. En pratique, prédire une **classe** (rapide/modéré/lent/timeout probable) est souvent plus robuste qu'une prédiction exacte en secondes.

11 Limites de l'approche

11.1 Limites théoriques

- Les formules proposées donnent des **ordres de grandeur**, pas des temps exacts.
- Les bornes asymptotiques ignorent les constantes, la structure fine des données et les effets d'implémentation.
- Les preuves de NP-difficulté portent sur des hypothèses explicites de modélisation (slots discrets, contraintes d'incompatibilité exprimables, etc.).

11.2 Limites pratiques

- Le comportement réel dépend fortement du solveur et du flattening.
- Deux instances de même taille peuvent avoir des temps très différents.
- Les solveurs utilisent des heuristiques complexes (branching, coupes, propagation, apprentissage), difficiles à modéliser analytiquement.
- Les performances doivent être validées expérimentalement sur un corpus représentatif.

12 Conclusion

Ce rapport établit que :

1. le parsing et la génération sont en général polynomiaux ;
2. la taille du modèle peut croître rapidement avec A, R, K, T ;
3. des familles comme le non-chevauchement peuvent atteindre $O(RA^2)$;
4. l'espace de recherche naïf est exponentiel ;
5. la faisabilité du planning est NP-difficile sous hypothèses raisonnables ;
6. l'optimisation est au moins aussi difficile que la satisfaction ;
7. le temps exact est difficile à prédire ;
8. un score de complexité et un modèle empirique sont utiles pour estimer le risque ;
9. les mesures expérimentales sont indispensables ;
10. le choix du solveur dépend fortement de la représentation MiniZinc effective.

La perspective opérationnelle est donc double : maintenir une analyse théorique rigoureuse pour guider la modélisation, et instrumenter la plateforme pour produire des données expérimentales robustes permettant une prédiction fiable des risques de lenteur ou de *timeout*.

13 Récapitulatif des résultats démontrés

1. $N_{\text{start}} = A$.
2. $N_{\text{assign}} = AR$.
3. $N_{\text{role}} = AKR$.
4. $N_{\text{variables}} = A + AR + AKR$.
5. $N_{\text{overlap}} = RA(A - 1)/2$.
6. $N_{\text{overlap}} \in O(RA^2)$.
7. $M = O(A + AR + AKR + RA^2 + RTA + Q + P)$.
8. $T_{\text{generation}} = O(M)$.
9. $S_{\text{generation}} = O(L + M)$ (souvent dominée par $O(M)$).
10. $E_{\text{search}} = T^A \times 2^{AR} \times 2^{AKR}$.
11. NP-difficulté par réduction depuis la k -coloration.
12. L'optimisation est au moins aussi difficile que la satisfaction.

14 Références

Références

- [1] M. R. Garey and D. S. Johnson, *Computers and Intractability : A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [2] K. R. Apt, *Principles of Constraint Programming*, Cambridge University Press, 2003.
- [3] F. Rossi, P. van Beek, and T. Walsh (eds.), *Handbook of Constraint Programming*, Elsevier, 2006.
- [4] K. Marriott, P. Stuckey, M. Garcia de la Banda, and M. Wallace, *The MiniZinc Handbook*, Springer, 2016.
- [5] MiniZinc Project, *MiniZinc Documentation*, <https://www.minizinc.org/>.
- [6] HiGHS Optimization, *HiGHS Documentation*, <https://highs.dev/>.
- [7] Gecode Team, *Gecode Documentation*, <https://www.gecode.org/>.
- [8] Chuffed Solver, *Chuffed Documentation/Repository*, <https://github.com/chuffed/chuffed>.
- [9] Google OR-Tools, *CP-SAT Solver Guide*, <https://developers.google.com/optimization>.